Publicación de notas:	14 de junio de 2023
Revisión:	16 de junio de 2023 de 10:30-12:30

FIRMA

Apellidos: _____

Nombre: _____

DNI: _____

PROBLEMA 3 (4 puntos)

Un equipo de ingenieros de software está desarrollando una plataforma web de comercio electrónico dedicada a la venta de diferentes tipos de productos. De cada producto se desea registrar en la plataforma su código (que debe ser único), su URL, su título, una descripción, su precio en euros, una imagen en la que aparezca claramente el producto y una serie de etiquetas. Cada producto deberá tener al menos una etiqueta (p. ej., "Dragon Ball Z"), siendo posible que una misma etiqueta sea asignada a varios productos diferentes. Una funcionalidad que se desea que tenga la plataforma de comercio electrónico es la de generar automáticamente un catálogo de productos en formato PDF. De acuerdo al prototipo desechable elaborado durante la fase de elicitación de requisitos, cada producto debe verse en el catálogo de la forma indicada en la Figura 1, la cual muestra cómo debería verse en el catálogo el producto titulado "Peluche Dragon Ball Z Serie 2: Kid Boo 15 cm". También se ha decidido que la plataforma de comercio electrónico cuente con un sistema de recomendación capaz de generar y mostrar recomendaciones de productos personalizadas para un usuario. Estas recomendaciones deben mostrarse de la forma indicada en la Figura 2, la cual muestra una imagen del prototipo desechable previamente mencionado. Además, si un usuario selecciona un producto recomendado, éste debe acceder a la URL de dicho producto.

Figura 1. Ejemplo de producto mostrado en el catálogo.

Figura 2. Ejemplo de recomendaciones de productos mostradas por el recomendador.

En la Figura 3 se muestra un fragmento del **diagrama de clases de diseño** que ha realizado el equipo de ingenieros de software para modelar la plataforma web de comercio electrónico en el cual se han representado únicamente las clases correspondientes a los siguientes tres componentes: "GestorProductos", "GestorCatalogos" y "GestorRecomendaciones".

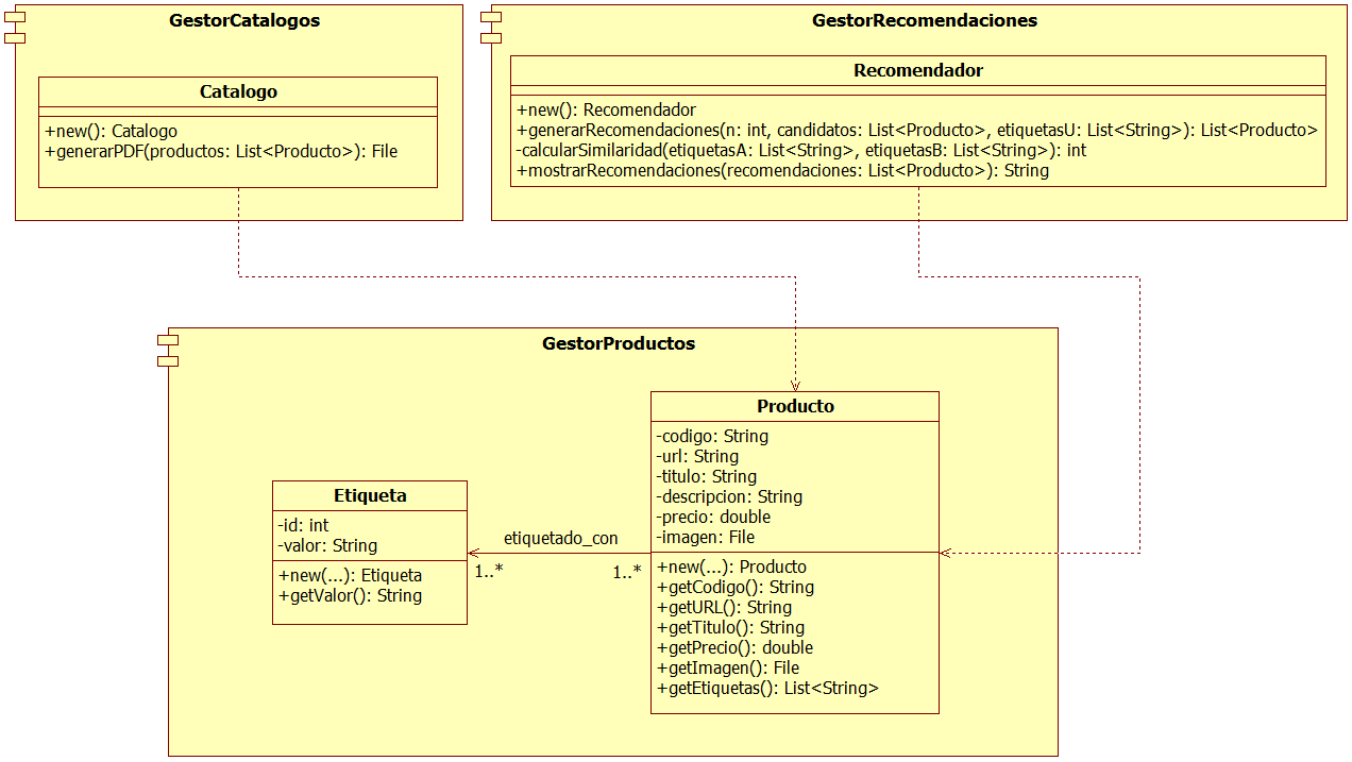


Figura 3. Diagrama de clases de diseño de la plataforma web de comercio electrónico.

El método "`getEtiquetas`" devuelve una lista con los valores de todas las etiquetas del producto. El método "`mostrarRecomendaciones`" devuelve el código HTML necesario para mostrar las recomendaciones en un navegador web y permitir a los usuarios acceder a la URL de los productos recomendados. El método "`generarRecomendaciones`" recibe tres parámetros: la cantidad de recomendaciones solicitadas (`n`), una lista de productos entre los cuales elegir los productos a recomendar (`candidatos`), y las preferencias del usuario para el cual se solicitan las recomendaciones expresadas en forma de etiquetas (`etiquetasU`). La implementación de este método se muestra a continuación:

```
public List<Producto> generarRecomendaciones(int n, List<Producto> candidatos, List<String> etiquetasU) {
  Collections.sort(candidatos, (c1, c2) -> {
    return (calcularSimilaridad(etiquetasU,c2.getEtiquetas()) – calcularSimilaridad(etiquetasU,c1.getEtiquetas()));
  });
  return candidatos.subList(0, Math.min(candidatos.size(),n));
}
```

El método "`generarRecomendaciones`" hace uso del método "`calcularSimilaridad`" para calcular, en una escala de 0 a 10, la similitud entre las preferencias del usuario y las etiquetas de cada producto que puede ser recomendado.

Apartado 3.1 (0,75 puntos)

Modifique el **diagrama de clases de diseño** proporcionado en el enunciado a fin de reducir el acoplamiento entre componentes lo máximo posible aplicando el principio de inversión de dependencias, pero sin aplicar el principio de segregación de interfaces. La solución propuesta **debe incumplir el principio de segregación de interfaces**.

Apartado 3.2 (0,75 puntos)

Modifique el **diagrama de clases de diseño** proporcionado en el enunciado a fin de reducir el acoplamiento entre componentes lo máximo posible **aplicando el principio de inversión de dependencias y el principio de segregación de interfaces**.

Apartado 3.3 (0,5 puntos)

Realice el **diagrama de componentes** de la plataforma web de comercio electrónico en base exclusivamente al diagrama de clases de diseño solicitado como solución del apartado anterior (es decir, del apartado 3.2).

Apartado 3.4 (1 punto)

Se desea que la plataforma web de comercio electrónico soporte dos nuevos tipos de productos: videojuegos y juegos de mesa. De estos productos la plataforma debe conocer, no solamente los datos generales (código, URL, título, descripción, precio, imagen y etiquetas), sino también una serie de datos adicionales. De cada videojuego se debe conocer su plataforma (p. ej., "PlayStation 5") y la edad recomendada de acuerdo a su código PEGI (p. ej., "12"), mientras que de cada juego de mesa se debe conocer la cantidad mínima y máxima de jugadores que pueden jugar. También se desea que el componente "GestorCatalogos" pueda generar catálogos con toda esta información (es decir, incluyendo la plataforma y edad recomendada de los videojuegos y la cantidad mínima y máxima de jugadores de los juegos de mesa) **sin conocer los diferentes tipos de productos existentes ni depender de los detalles de implementación de "GestorProductos"**. Elabore, partiendo del diagrama de clases de diseño proporcionado en el enunciado, un **diagrama de clases de diseño** que contenga únicamente las clases pertenecientes a los componentes "GestorProductos" y "GestorCatalogos" y que represente un diseño de estos componentes que satisfaga los nuevos requisitos. Posteriormente, indique el **nombre de los principios de diseño aplicados** en el recuadro inferior.

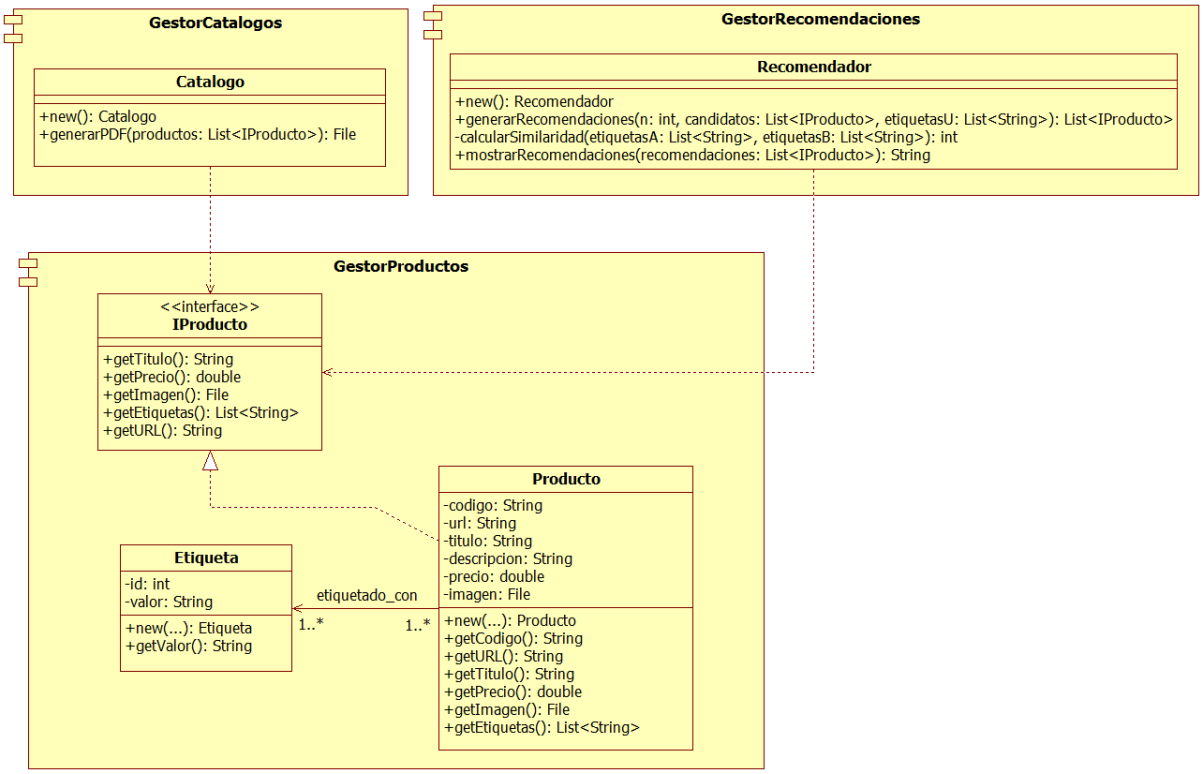
Principios de diseño aplicados:

Apartado 3.5 (1 punto)

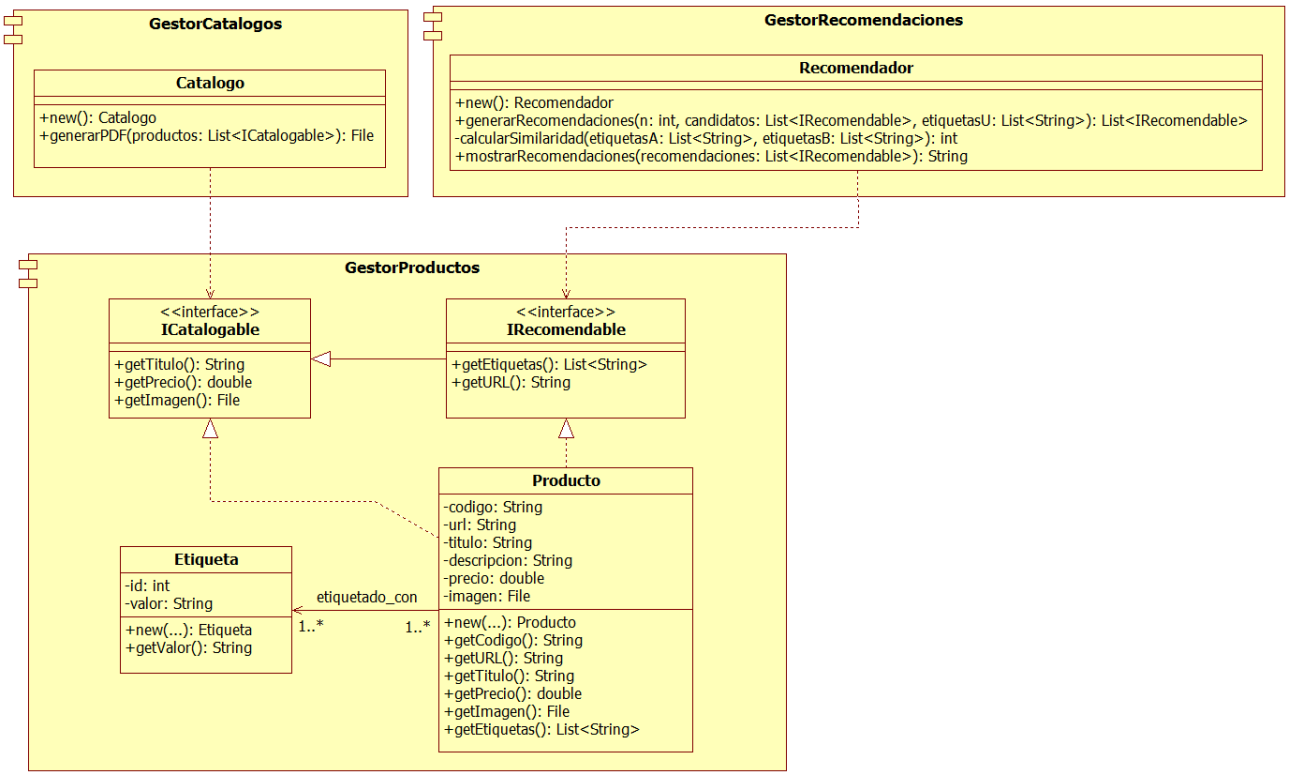
Se desea que el componente “GestorProductos” permita a los administradores de la plataforma web de comercio electrónico dar de alta nuevos productos, listar todos los productos dados de alta y consultar y eliminar un producto concreto que ha sido previamente dado de alta. Elabore, partiendo del diagrama de clases de diseño proporcionado en el enunciado y empleando el patrón arquitectónico **MVC (Modelo-Vista-Controlador)**, un **diagrama de clases de diseño** que contenga únicamente las clases pertenecientes al componente “GestorProductos” y que represente un diseño de este componente que satisfaga estos nuevos requisitos.

Solución

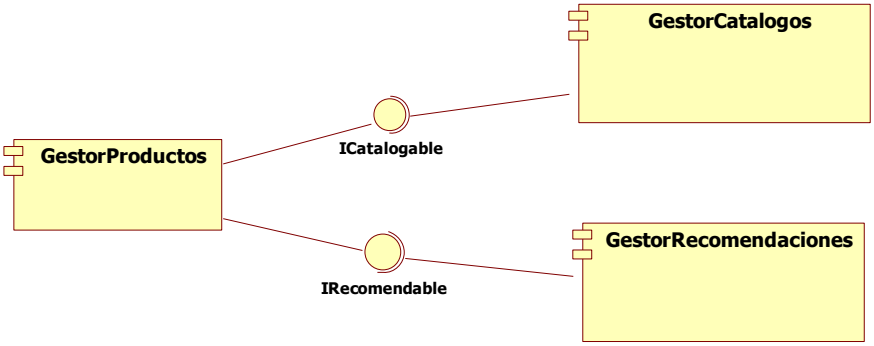
Apartado 3.1



Apartado 3.2

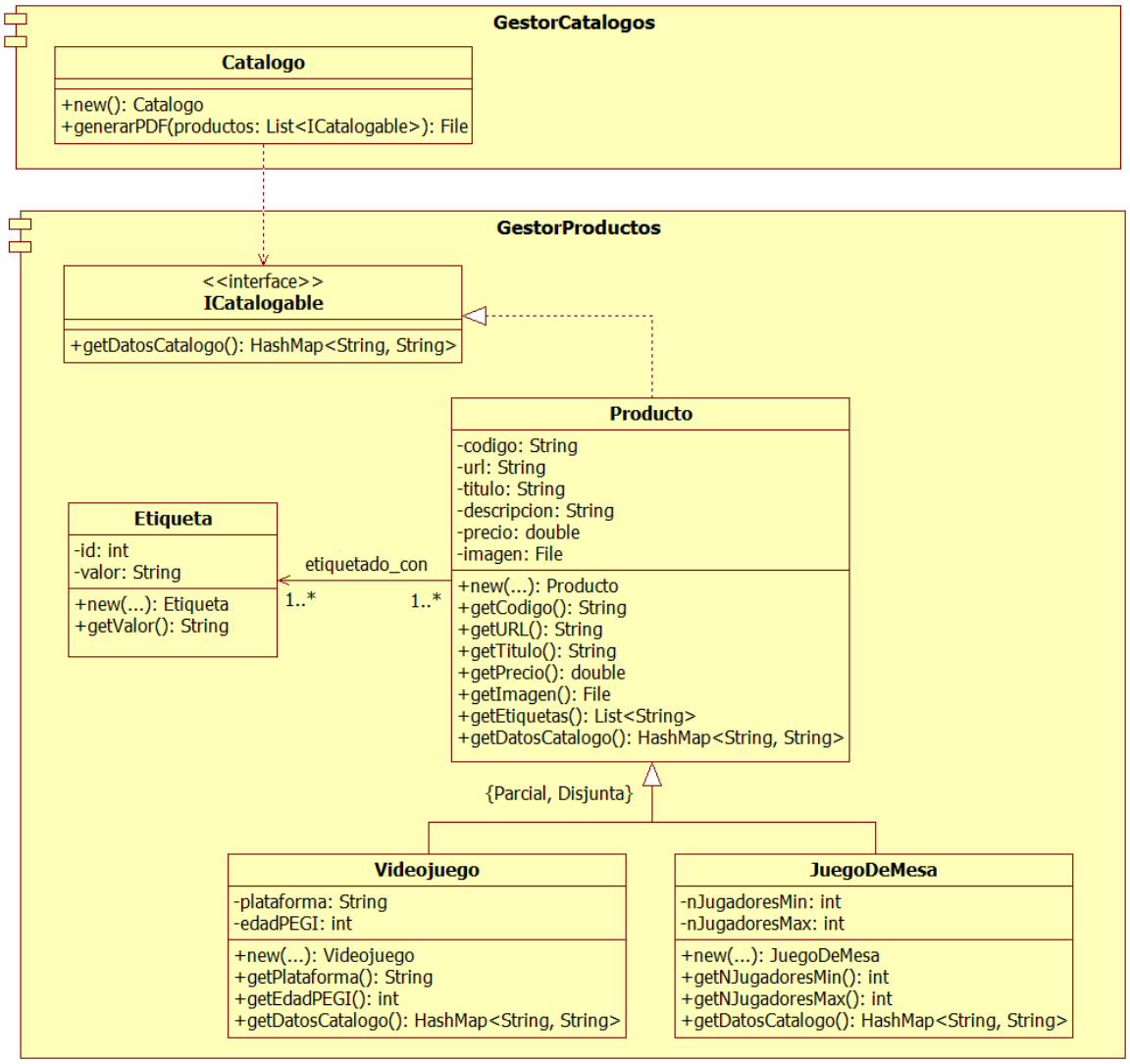


Apartado 3.3

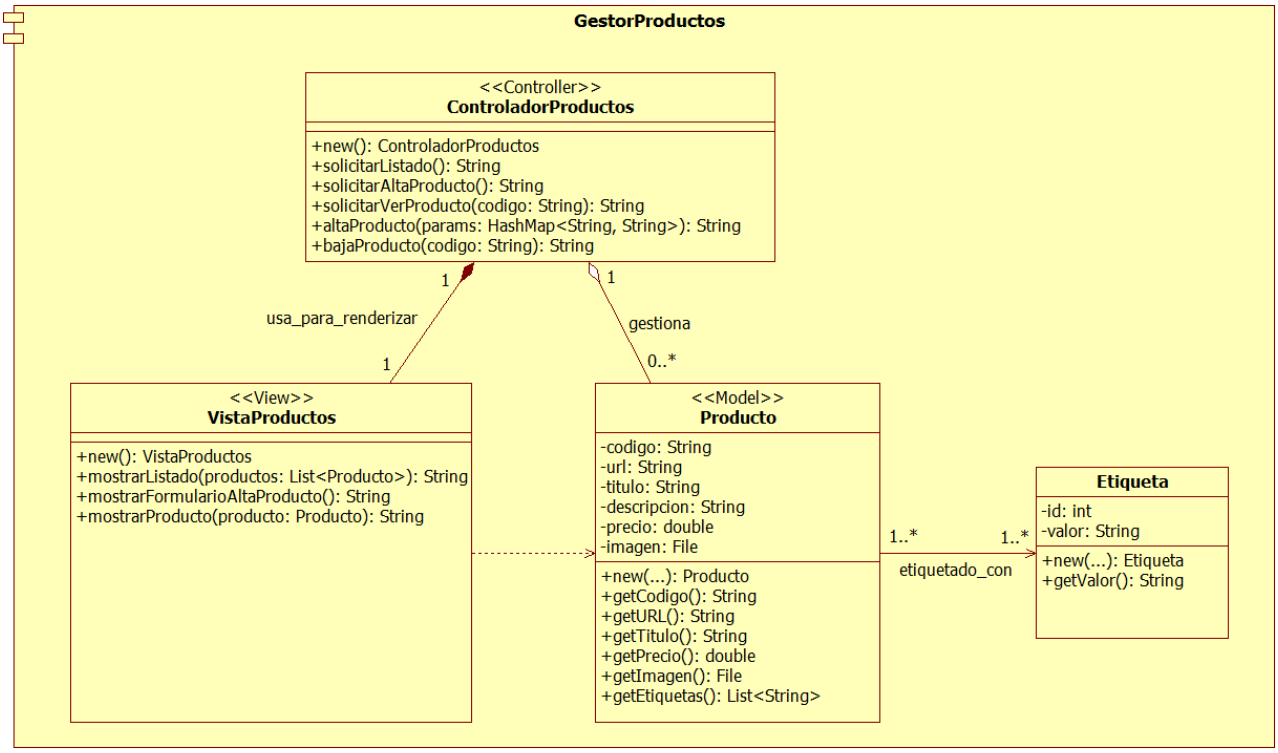


Apartado 3.4

Principios de diseño aplicados: Abstracción por especialización, abierto/cerrado, sustitución de Liskov e inversión de dependencias.



Apartado 3.5


GestorProductos
<<Controller>>
ControladorProductos
+new(): ControladorProductos
+solicitarListado(): String
+solicitarAltaProducto(): String
+solicitarVerProducto(codigo: String): String
+altaProducto(params: HashMap<String, String>): String
+bajaProducto(codigo: String): String
1
usa_para_renderizar
1
1
gestiona
0..*
<<View>>
VistaProductos
+new(): VistaProductos
+mostrarListado(productos: List<Producto>): String
+mostrarFormularioAltaProducto(): String
+mostrarProducto(producto: Producto): String
<<Model>>
Producto
-codigo: String
-url: String
-titulo: String
-descripcion: String
-precio: double
-imagen: File
+new(...): Producto
+getCodigo(): String
+getURL(): String
+getTitulo(): String
+getPrecio(): double
+getImagen(): File
+getEtiquetas(): List<String>
1..*
1..*
etiquetado_con
Etiqueta
-id: int
-valor: String
+new(...): Etiqueta
+getValor(): String